

Lecture 33: Vision-Language Models I

How a model sees

The most ordinary thing you do with a chat model

You paste a photo of a handwritten receipt and ask what the total is. It answers correctly. You paste a chart and ask which quarter dropped. It answers.

Nothing in this course so far can do that.

A transformer (ch9) consumes a **sequence of vectors**. Text arrives as one naturally: split into tokens, look each up in an embedding table.

An image is a **grid of pixels**. There is no sentence, no vocabulary, and no obvious order. Nothing to look up.

This lecture is that bridge. The next one is the harder direction: making the same model **draw**.

Two questions that sound the same and are not

1. How does a model see an image?

Settled. One recipe, essentially everyone uses it, and the arguments left are about efficiency.

This lecture. You will be able to explain every open-weights vision-language model released in the last three years.

2. How does a model draw an image?

Not settled. Three competing architectures, actively argued about, and the leading products do not agree with each other.

L34. The disagreement is the interesting part, and it comes from a genuine conflict with what you built in ch10.

A **vision-language model (VLM)** is any model that takes both images and text as input. The good ones do both questions at once.

Outline

An image is not a sequence

Two encoders, one space

Three ways to give an LLM eyes

What it costs, and what breaks

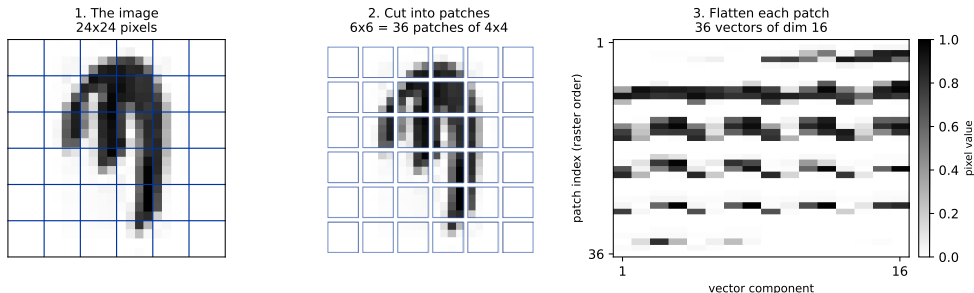
An image is not a sequence

So make it one. The trick is almost insultingly simple.

Patchify

Cut the image into fixed squares. Flatten each square into a vector. Push each vector through **one linear layer**. Done - the image is now a sequence, and every tool from ch9 applies unchanged.

An image becomes a sequence of vectors - and a transformer can read it



Shown on a letter from ch10, at 24×24 with 4×4 patches. Each patch becomes a 16-dimensional vector; the linear layer maps it to whatever width the transformer wants.

The arithmetic that governs everything downstream

A patch grid of side s/p gives $(s/p)^2$ tokens. This number comes back as a **cost** in every design decision for the rest of the lecture.

Resolution	Patch	Grid	Tokens
224×224	16×16	14×14	196
224×224	14×14	16×16	256
336×336	14×14	24×24	576
896×896	14×14	64×64	4096

The highlighted row is **LLaVA-1.5** (Liu et al., 2023), the reference open vision-language model. Remember **576**.

Tokens grow with the *square* of resolution, and attention costs the *square* of tokens. Doubling resolution is roughly $16\times$ the attention work.

The same thing, on a real photograph

A 24×24 letter makes the idea clear but hides the scale. Here is what a real vision encoder actually does to a real image.

224×224 , the standard input



cut into 16×16 patches
 $14 \times 14 = 196$ tokens

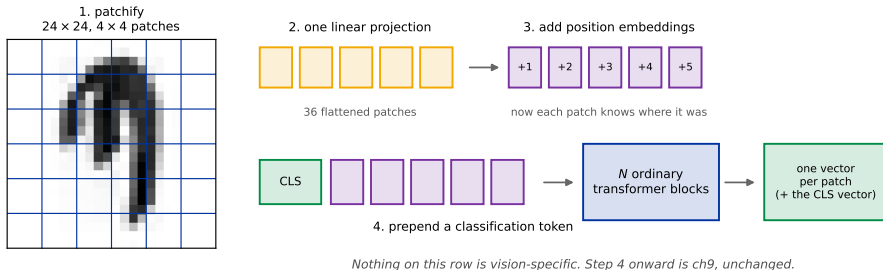


Each square becomes ONE vector. The transformer never sees a pixel grid again.

A market in Yerevan. Keep this picture in mind - by the end of the lecture it will have broken three different things.

The Vision Transformer

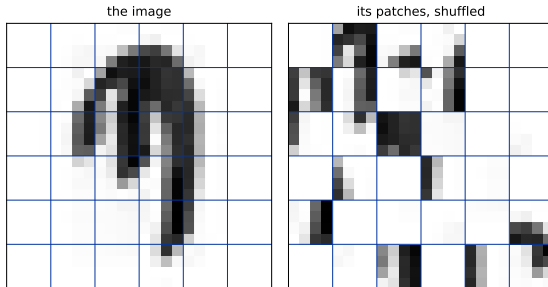
Dosovitskiy et al. (2021) put patches into a standard transformer encoder and changed almost nothing else. The **Vision Transformer (ViT)** is the result, and it underlies essentially every vision encoder in use today.



Step 4's **classification token (CLS)** is an extra learned vector whose output summarises the whole image. **The naming is honest:** nothing in this diagram is vision-specific - it is the ch9 architecture, fed differently.

Why step 3 is not optional

Self-attention is **permutation-equivariant**: shuffle the inputs and you get the same outputs, shuffled. It has no built-in idea of "next to".



To self-attention,
without position embeddings,

**these two are
THE SAME INPUT**

Attention is permutation-equivariant:
it sees a SET of patches, not a grid.

A **convolution** could never make this mistake - locality is wired into the kernel. A transformer has to be *told* where each patch was, by adding a learned vector per position. Take that away and an image is a bag of unordered squares.

Predict first

A CNN (ch6) is built around a strong assumption: **nearby pixels belong together**, hard-wired by the convolution kernel. A Vision Transformer has no such wiring - every patch can attend to every other patch from layer one.

So: does a Vision Transformer need convolution to learn that neighbouring patches are related?

Predict first

A CNN (ch6) is built around a strong assumption: **nearby pixels belong together**, hard-wired by the convolution kernel. A Vision Transformer has no such wiring - every patch can attend to every other patch from layer one.

So: does a Vision Transformer need convolution to learn that neighbouring patches are related?

No - given enough data. With large training sets it learns local structure by itself, and beats comparable CNNs.

But on small data it loses. The convolution's assumption is genuinely correct, and a model given the right prior needs fewer examples to find it.

This is the bias-variance tradeoff from ch2, wearing an architecture costume: a helpful constraint is worth a lot when data is scarce and becomes a ceiling when it is not.

Two encoders, one space

Patches are only useful if their meaning lines up with words.

The problem with a plain Vision Transformer

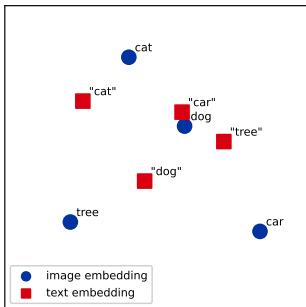
Train a Vision Transformer to classify 1000 categories and its features are excellent **at those 1000 categories**. Ask about anything else and the representation was never asked to encode it.

We want an image representation shaped by **language**, so that "a photo of a dog wearing a hat" and the corresponding picture land in the same place. Then a language model can read the image features because they already speak its language.

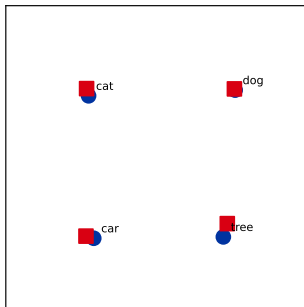
The supervision for that is lying around the internet in enormous quantity: images that came with captions.

What "one space" actually means

before training: two unrelated spaces



after training: paired things point the same way



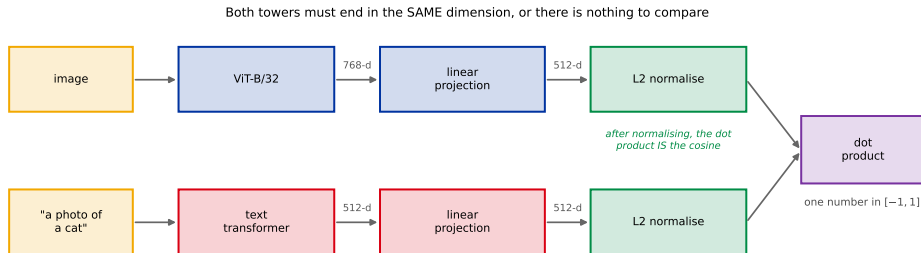
Schematic - a real space has 512 dimensions, not 2

Two encoders, one destination. A picture of a cat and the string "a cat" should end up pointing the same way, **even though no shared weight ever processed both**.

Once that holds, an image is **addressable by language** - which is exactly what a language model needs to read it. The rest of this section is how to get from the left panel to the right one.

CLIP: two towers that have to meet

Contrastive Language-Image Pre-training (CLIP), Radford et al. (2021). Two completely separate encoders, trained together on **400 million** image-caption pairs scraped from the web.



The two towers share **no weights** and never see each other's inputs. The only thing forcing them to agree is the loss - which is why the last two boxes matter so much.

Two details in that diagram that are easy to skip

The linear projection

A Vision Transformer might output 768 numbers; the text transformer 512. You cannot compare vectors of different lengths at all.

So each tower ends with one learned matrix mapping into a **shared embedding space** - 512 dimensions for CLIP ViT-B/32. That space is the whole product. Everything else is a way of shaping it.

The L2 normalisation

Every embedding is divided by its own length, so it lands on the unit sphere. Now the dot product of two embeddings is exactly the **cosine of the angle** between them.

Meaning becomes **direction**, not magnitude. A picture of a cat and the words "a cat" should point the same way; how long the arrows are means nothing.

The objective, written out

Take a batch of N pairs. Embed everything and build the $N \times N$ matrix of cosine similarities s_{ij} between image i and caption j .

Read **row** i as a multiple-choice question: "*which of these N captions belongs to image i ?*" The right answer is always on the diagonal - so this is ordinary classification with N classes, and takes ordinary cross-entropy:

$$\mathcal{L}_{\text{image} \rightarrow \text{text}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^N \exp(s_{ij}/\tau)}$$

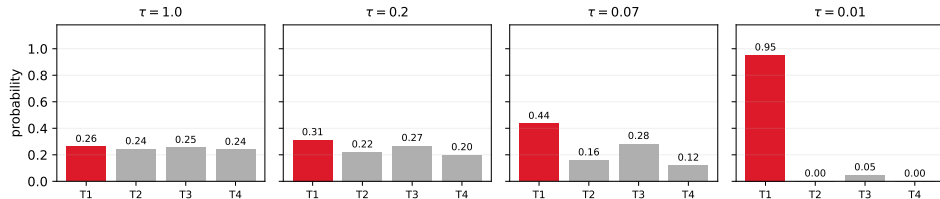
Then again down the **columns** - "which image belongs to this caption?" - and average the two. The loss is **symmetric**; neither modality is the label.

τ is the **temperature**, and CLIP does not fix it. It is learned by gradient descent like any other weight.

Temperature: how confident should the model be?

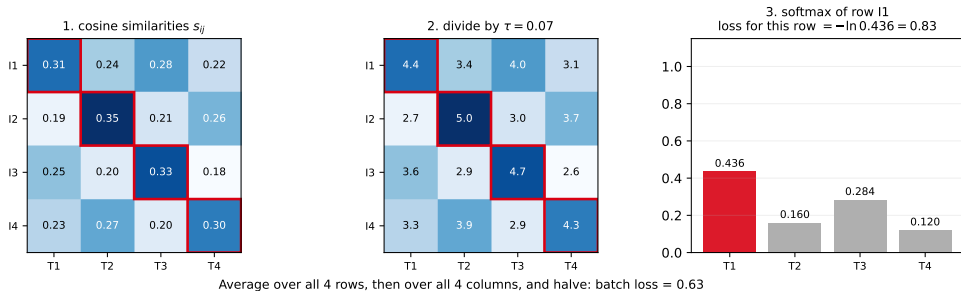
Cosine similarities live in $[-1, 1]$ and in practice bunch together. Feed those straight into a softmax and every answer looks equally good, so there is almost no gradient. Dividing by a small τ spreads them apart.

Same four similarities (0.31, 0.24, 0.28, 0.22), four temperatures. CLIP initialises τ at 0.07 and learns it.



τ controls **how harshly a near-miss is punished**. CLIP does not tune it by hand: it is **initialised at 0.07 and learned**, so the model decides for itself how sharp its own softmax should be - one fewer hyperparameter, and a rare case of a training difficulty setting being handed to gradient descent.

By hand: one training step on a batch of 4

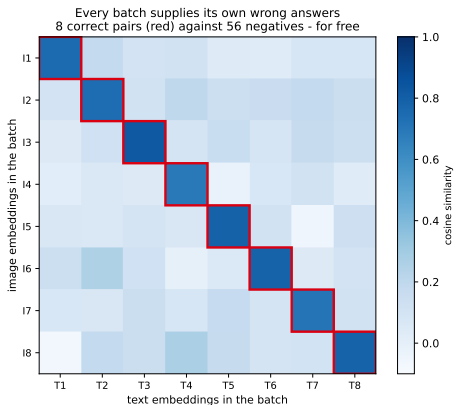


Row I1's correct caption scored 0.31; the best wrong one scored 0.28. After dividing by τ that gap becomes 4.4 against 4.0, and the softmax puts **0.436** on the right answer, for a row loss of $-\ln 0.436 = 0.83$.

A perfect batch scores 0; random guessing on 4 classes scores $\ln 4 = 1.39$. At **0.63** this model beats chance and has plenty left to learn.

Why the free negatives matter so much

Supervised learning needs someone to write down the right answer. Contrastive learning only needs pairs that **belong together**, and it manufactures its own wrong answers from the rest of the batch.



At $N = 32,768$, one step supplies about **one billion** negative comparisons for the cost of embedding 32,768 items.

Batch size is not a speed knob here. A bigger batch means more wrong answers to rule out, so the question is genuinely harder and the features have to be sharper. Small batch, easy task, weaker model.

Illustrative values; the **structure** is exact.

The catch, and the fix everyone now uses

That batch-wide softmax is expensive in a specific way: normalising a row needs every other device's embeddings, so the loss does not shard cleanly across machines.

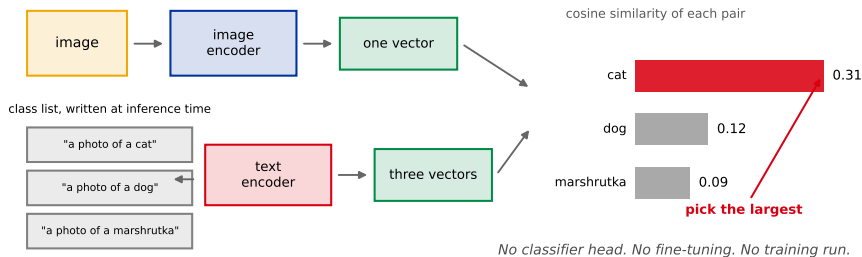
SigLIP (Zhai et al., 2023), **Sigmoid Loss for Language-Image Pre-training**, replaces the softmax over a whole row with an **independent sigmoid on each pair**: each of the N^2 cells becomes its own yes/no question.

No normalisation across the batch, so it shards trivially - and it works well at *small* batch sizes, where CLIP's objective degrades.

It is now the default vision encoder in a large share of open vision-language models.

Zero-shot classification falls out for free

CLIP was never trained to classify anything. But it can: write each class as a *sentence*, embed them all, embed the image, and take the highest similarity.



The last step is the same dot product as in training - **classification and retrieval are the same operation here**, you have only chosen what to put in the columns. The class list is a **string**, decided at inference time.

Why "a photo of a" is not decoration

The prompt template looks like superstition. It is not, and the reason recurs everywhere in this course.

CLIP's text encoder never saw bare nouns. It saw **captions** - "a photo of a cat sitting on a windowsill". Feeding it the single word "*cat*" asks it to embed something **off the distribution it was trained on**, and the embedding is correspondingly worse.

Wrapping the class in a caption-shaped sentence puts it back on distribution. Then go further: embed the *same* class under **80 different templates** - "a bad photo of a...", "a sculpture of a..." - and average the resulting vectors.

Radford et al. report **3.5%** on ImageNet from that ensembling, and almost **5%** from prompt engineering and ensembling together. It is free at inference: average once, keep one vector per class.

What CLIP achieved, and where it stops

The headline result. Zero-shot CLIP matched a **ResNet-50 trained directly on ImageNet**, without seeing one ImageNet training image.

And it was far more **robust**: on distorted or unusual renderings of the same classes, the supervised model degraded sharply and CLIP largely held up. Learning from captions rather than a fixed label set gives a less brittle representation.

Bounded by the captions it saw. Concepts rare on the web - specialist medical imaging, minority scripts, fine-grained species - stay weak. "Zero-shot" means "no task training", not "knows everything".

It is also poor at **counting**, at reading text in images, and at spatial relations. Captions rarely say "three cups to the left of the laptop", so nothing ever taught it to encode that.

Hold on to the right-hand box. Every one of those weaknesses returns at the end of this lecture as a **vision-language model** failure - this is where they are born.

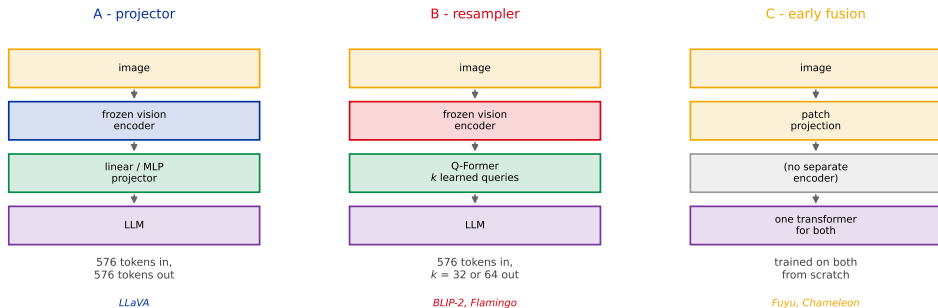
Three ways to give an LLM eyes

We have image tokens that mean something. Now attach them to a language model.

The three answers, side by side

Everyone agrees on the vision encoder. The disagreement is about **what sits between it and the language model** - and the real question underneath is how many tokens the language model is asked to swallow.

Same problem, three answers: how many tokens does the language model see, and who decides?



The next three frames take these one at a time. Watch the third row of each column.

Design A - a projector

The simplest thing that works, and it works remarkably well. **LLaVA** (Liu et al., 2023).



text tokens go in here too

A **multi-layer perceptron (MLP)** maps each of the 576 visual vectors into the language model's embedding space. They are then simply **prepended to the text tokens**, exactly as if they were words.

The language model is not modified at all. No new layers, no architecture change. That is why this took off - anyone could bolt vision onto any open LLM.

Cost: all 576 tokens sit in the context window for the whole conversation.

Design B - a resampler

If 576 tokens is too many, **learn to compress them** to a fixed budget. **Flamingo** (Alayrac et al., 2022) and **BLIP-2** (Li et al., 2023).

A small set of **learned query vectors** cross-attends to the image features and outputs one vector each. The output count is **constant** no matter how many patches went in - the image is summarised, not passed through.

Resolution and token cost are now **decoupled**. Feed a bigger image, pay the same downstream price. This is what makes video remotely affordable.

A fixed budget is a fixed budget. Whatever those queries fail to pick up is gone. Dense text in an image suffers badly.

A detail worth stealing. Flamingo injects vision through gated cross-attention whose gate is initialised to **zero**. At step one the vision path contributes exactly nothing, so the model *is* the original language model, and vision fades in as the gate opens. Adding a modality can therefore never damage what already worked - the same idea as a residual branch starting at zero.

Design C - early fusion

Remove the bridge entirely. **One transformer**, both modalities, from the first layer.

Chameleon (Meta, 2024) is the clearest example, and L34 is mostly about this family.

There is no adapter to train, no frozen encoder, no projection. Images and text are just two kinds of token in one vocabulary, and one loss covers both.

This is the only design that can **generate** images as naturally as it reads them. A projector is a one-way street.

Costs: you cannot reuse a pretrained language model as-is, training is much more expensive, and mixed-modality training is unstable enough that Chameleon needed specific fixes to converge at all.

Which won

	A: projector	B: resampler	C: early fusion
Token cost	full (576)	fixed, small	full
Pretrained LLM reusable	yes	yes	no
Training cost	lowest	low	highest
Can generate images	no	no	yes
Detail preserved	high	lossy	high

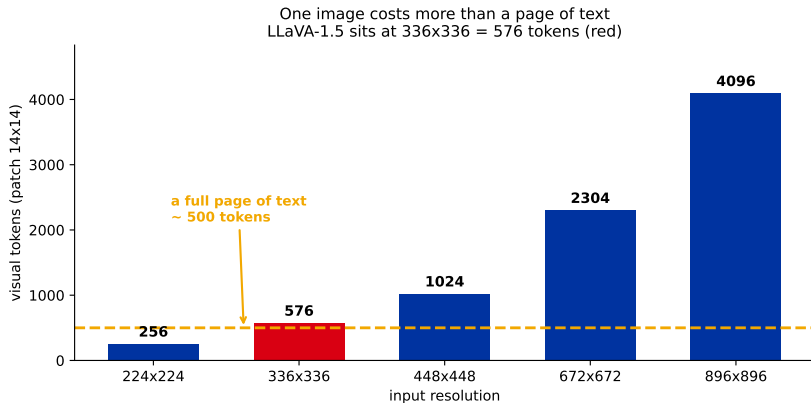
It depends on whether the model must also draw. For understanding alone, **A still dominates** - Qwen2-VL, InternVL and the LLaVA family all keep a separate encoder and a projector. **C is what you need the moment the model must generate images**, which is why every unified model in L34 is early-fusion. B survives where the token budget binds, above all in video.

So if you are building something today with an open model and a modest budget, A is the right default
- and that is not a compromise, it is the current state of the art for reading images.

What it costs, and what breaks

Every failure in this section traces back to one number.

An image is expensive



A 336×336 thumbnail costs more context than a full page of writing. A phone photo at native resolution would cost more than this entire lecture as text.

What the resize actually destroys

Back to the market. Those handwritten price labels are perfectly legible in the photograph. Here they are before and after the resize every fixed-resolution encoder performs:

the labels, at the photo's own resolution



the same labels after a 336 x 336 resize



576 tokens is not the constraint that hurts here - the RESIZE is. The writing is gone before the model starts.

No amount of cleverness downstream recovers this. The model is not failing to read the labels - it never received them. When a vision-language model "cannot read" a receipt, this is usually why, and it happens before layer one.

So stop resizing everything to a square

the original
960 × 720



squashed to 336 × 336
576 tokens, and distorted



4 crops at full detail + 1 thumbnail
5 × 576 = 2,880 tokens



Cut the image into crops, encode each at full resolution, and add one downscaled copy of the whole thing so the model still knows how the pieces fit together. The token bill goes up in exact proportion - here 5×.

The general version of that fix

The fixed-resolution habit is inherited from image classification, and it is actively harmful for documents: a wide receipt squashed to 336×336 has unreadable text **before the model sees it**.

Naive Dynamic Resolution (Qwen2-VL, Wang et al., 2024): keep the original aspect ratio, emit however many tokens the image actually needs. A small icon costs few tokens; a dense page costs many.

This single change is most of why recent models became usable for **screenshots, documents and charts**, which is now the dominant real-world use.

The active research direction is the other side of the same coin: **token pruning and merging** - drop or combine patches that carry little information, since most of a photograph is background.

Three failures, in one photograph

The market picture from the start of the lecture is a near-perfect adversarial example, and you can now explain every part of it.

Ask it...	and it struggles, because
"How many bowls of spice are there?"	dozens of near-identical objects, and attention <i>pools</i> rather than counts.
"What does that hand-written label say?"	the resize destroyed the strokes two slides ago - the model never received them.
"What script is that?"	Armenian handwriting is thin on the caption web CLIP learned from, so the representation is weak where the data was.

None of these is a mysterious model quirk. Each is a specific decision made earlier in this lecture, coming back with a bill.

Where vision-language models still fail

These are not random weaknesses. Each has a cause you can now name.

Failure	Why
Counting objects	Attention pools information; nothing in the training objective ever rewarded keeping a tally.
Precise spatial relations	Position embeddings give rough layout, not geometry.
Reading clocks and gauges	Needs exact angles from a coarse patch grid.
Dense or small text	Below the patch resolution the characters are simply not there.

Notice the pattern: most of these are the **token budget** again. The information was destroyed at the encoder, and no amount of language-model capability downstream can restore it. Same lesson as ch10, where halving a 1-2 pixel stroke twice erased it before the deep layers ever saw it.

Hallucination, in the visual case

A vision-language model will confidently describe objects that are not in the picture - most often objects that *usually* appear alongside what is there.

Ask about a kitchen photo and you may be told about a refrigerator that is not in frame. Kitchens have refrigerators, and the language model knows that far more strongly than it trusts 576 blurry vectors.

The language prior overrides the image. It is the *language* model that was trained on trillions of tokens; the visual evidence is comparatively thin and easy to outvote.

Practical consequence: a fluent, specific description is not evidence that the model looked carefully. If the answer matters, ask about what is *absent*, or ask twice with the object named differently.

Recap

1. Cut an image into patches, project each one, and it is a sequence a transformer can read. That is the whole bridge.
2. A Vision Transformer is the ch9 architecture, fed differently. Position embeddings are what stop it being a bag of patches.
3. CLIP aligns images and text by contrast, using the batch itself as negatives. Zero-shot classification falls out for free.
4. Attach it to a language model with a projector, a resampler, or no bridge at all. For reading images the projector still wins; the bridge only disappears when the model must also draw.
5. One 336×336 image costs **576** tokens, and most failures trace back to that number.

Next: reading an image is settled. **Drawing** one from the same model is not.

L34 starts from a real conflict: your ch10 diffusion model works in continuous space, and a language model emits discrete tokens. Three different architectures resolve that contradiction three different ways, and we will measure what the most popular one costs on our own letters.